

DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 2:

Storage Components

(part_1)

Storage components

- Storage components store data and perform some simple operations.
- Storage components include:
 - registers
 - counters
 - memories
 - stacks
 - queues
- Combinatorial and storage components are used for construction of:
 - datapaths
 - controllers
- Main subsystems of modern processors and other microchips.

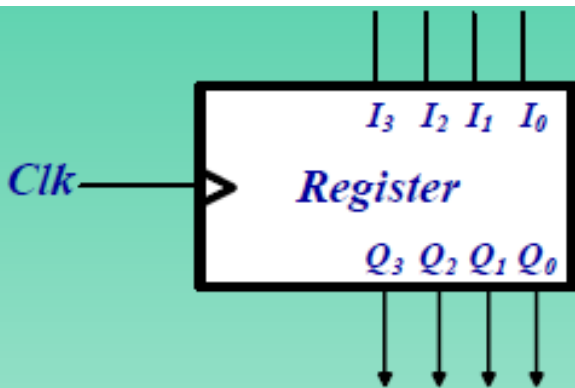
Storage components

- Storage components store data and perform some simple operations.
- **Storage components include:**
 - **registers**
 - **counters**
 - **memories**
 - **stacks**
 - **queues**
- Combinatorial and storage components are used for construction of:
 - datapaths
 - controllers
- Main subsystems of modern processors and other microchips.

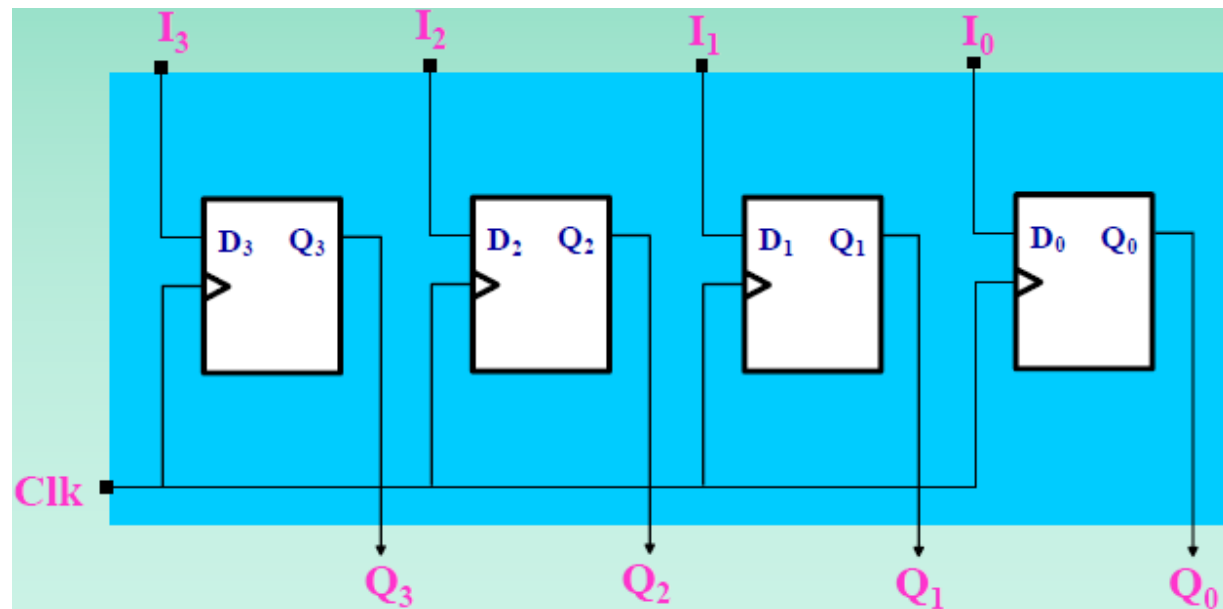
Registers

- Registers are bit wise extensions of flip-flops.
- Registers store one data word.

$$Q_i = I_i \text{ when Clk} = \uparrow$$



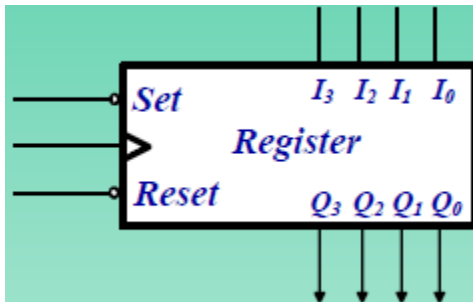
Graphic symbol



Register schematic

Registers with asynchronous set and reset

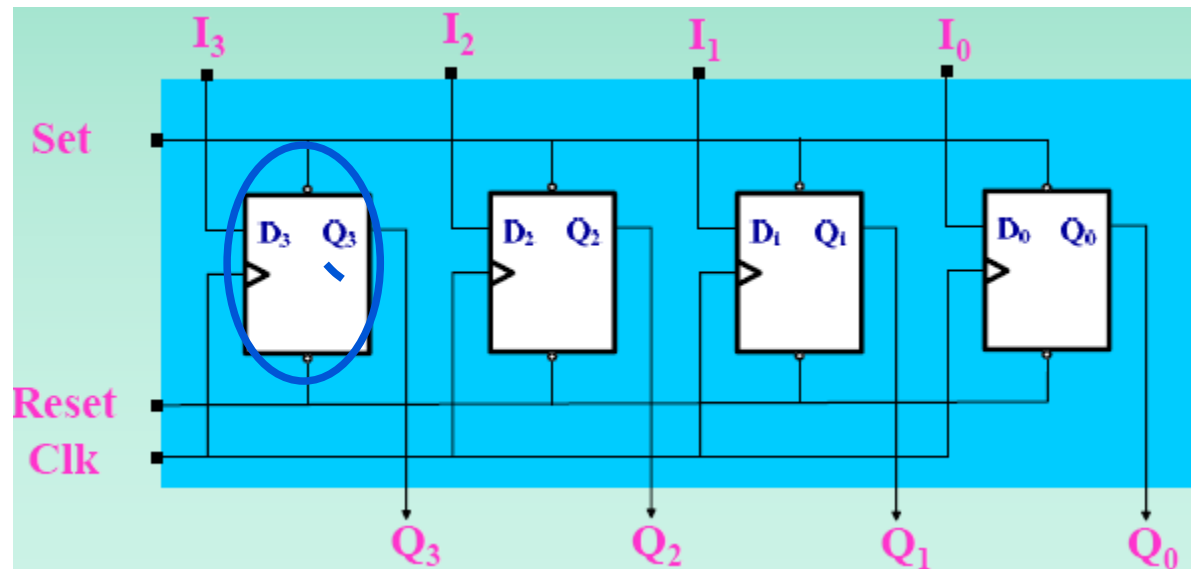
- Asynchronous setting and resetting is independent of clock signal.
- Synchronous inputs are used to initialize the register.



Graphic symbol

S	R	Clk	Q_i
0	X	X	1
X	0	X	0
1	1	↑	I_i

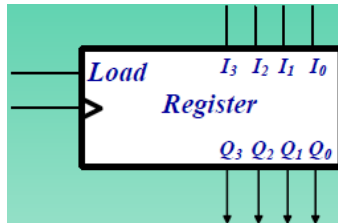
Operation table



Register schematic

Register with parallel load

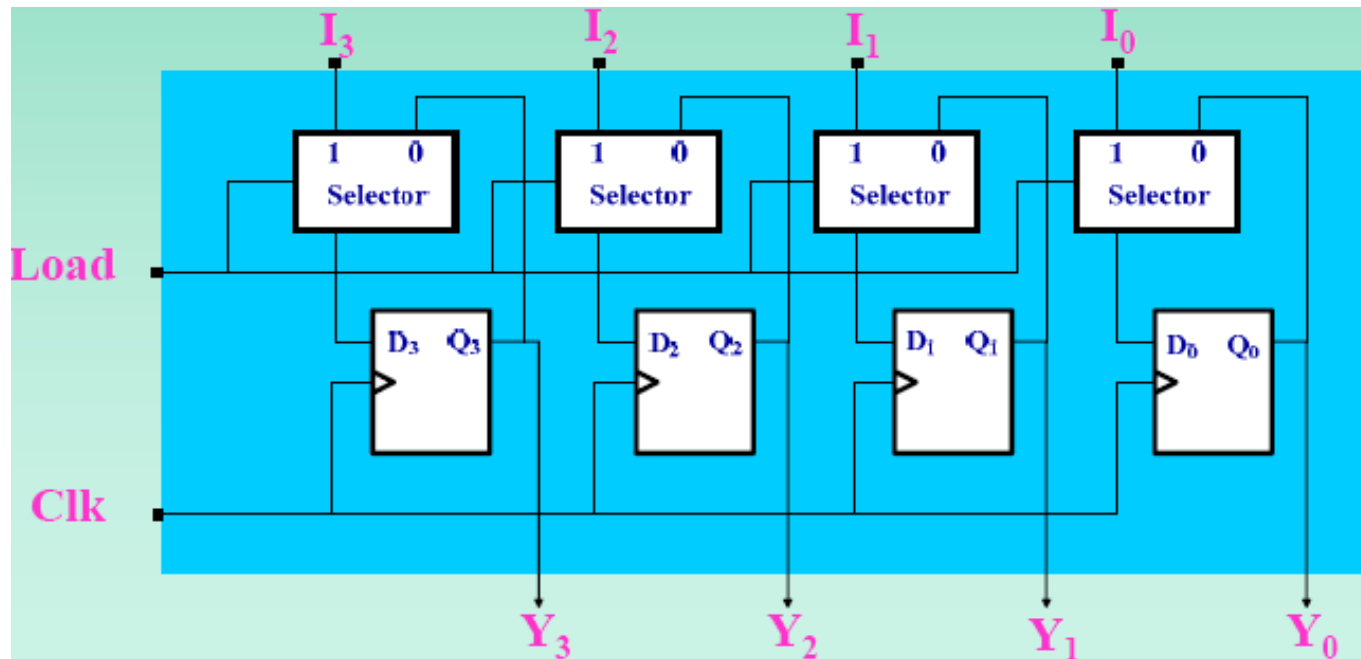
- Parallel load register can hold data indefinitely.
- It can also load new data when load signal is 1.



Graphic symbol

Present state	Next state
Load	$Q_3 Q_2 Q_1 Q_0$
0	No change
1	$I_3 I_2 I_1 I_0$

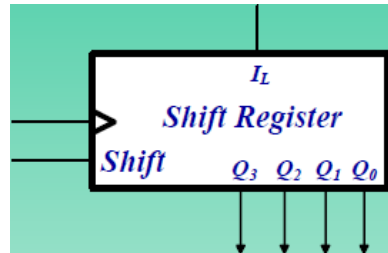
Operation table



Register schematic

Serial-in, parallel-out Shift register (shift-right)

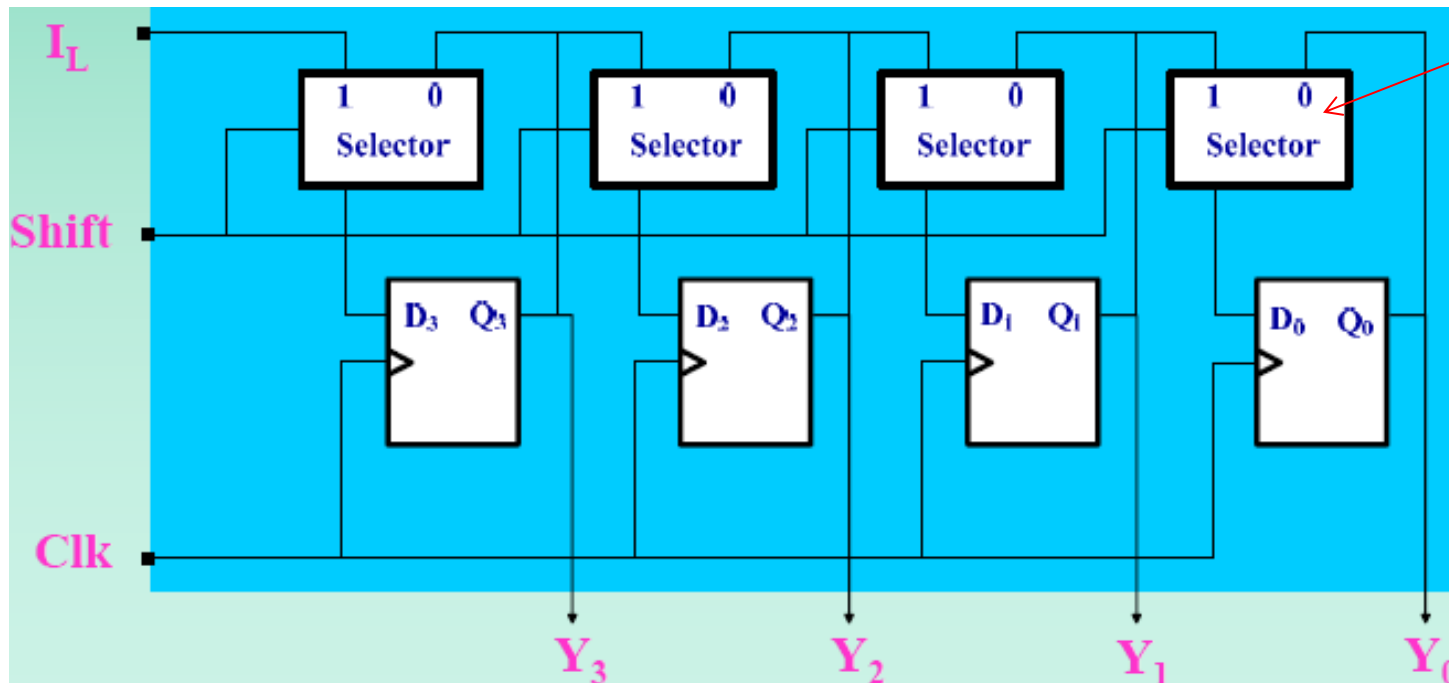
- Serial-in, parallel-out register converts serial data stream into parallel data stream.



Graphic symbol

Present state	Next state
Shift	$Q_3 Q_2 Q_1 Q_0$
0	No change
1	$I_L Q_3 Q_2 Q_1$

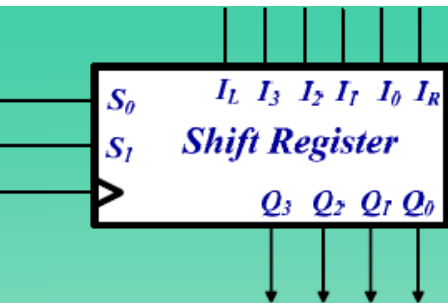
Operation table



Mux 2-to-1

Register schematic

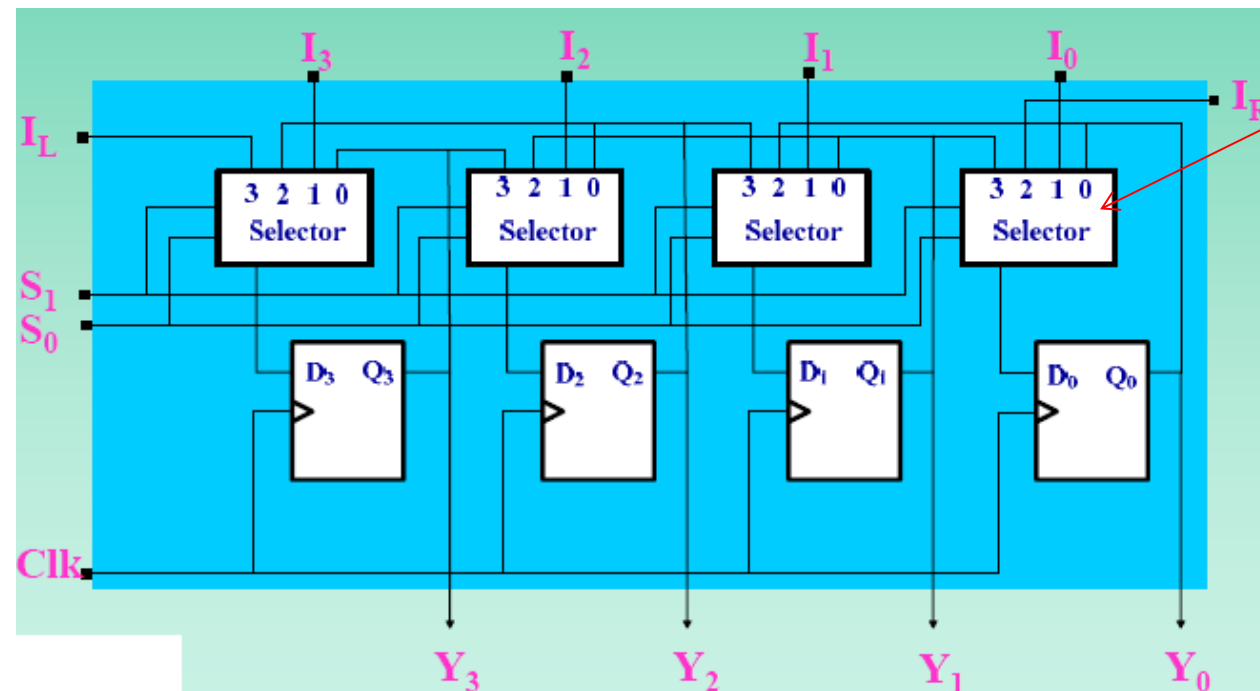
Shift register with parallel load (SRwPR)



Graphic symbol

Present state		Operation	Next state			
S_1	S_0		Q_3	Q_2	Q_1	Q_0
0	0	No change	Q_3	Q_2	Q_1	Q_0
0	1	Load input	I_3	I_2	I_1	I_0
1	0	Shift left	Q_2	Q_1	Q_0	I_R
1	1	Shift right	I_L	Q_3	Q_2	Q_1

Operation table



Register schematic

Mux 4-to-1

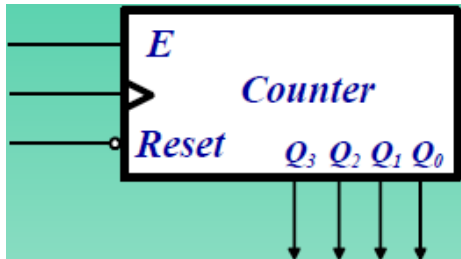
$$D_0 = S_1' S_0' Q_0 + S_1' S_0 I_0 + S_1 S_0' I_R + S_1 S_0 Q_1$$

$$D_i = S_1' S_0' Q_i + S_1' S_0 I_i + S_1 S_0' Q_{i-1} + S_1 S_0 Q_{i+1} \quad (1 \leq i \leq 2)$$

$$D_3 = S_1' S_0' Q_3 + S_1' S_0 I_3 + S_1 S_0' Q_2 + S_1 S_0 I_L$$

4-bit synchronous binary counter

- Counters increment (decrement) their content when enabled



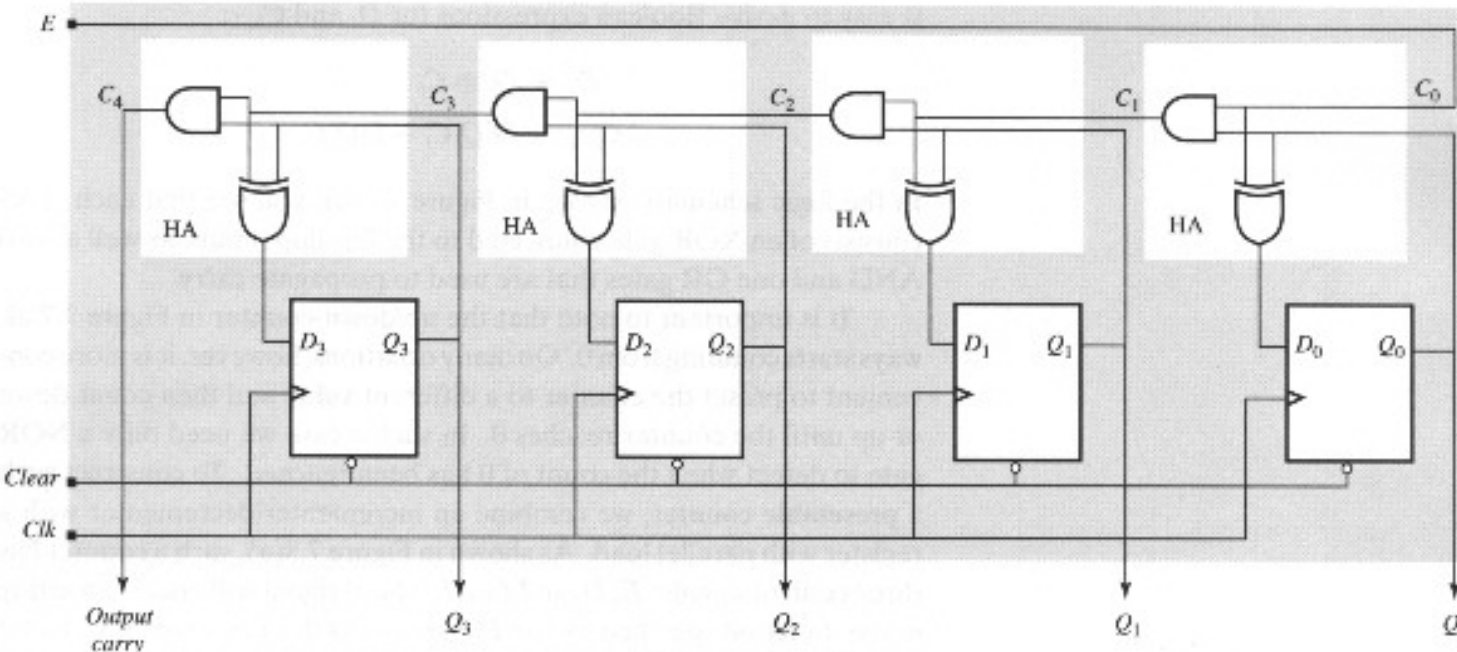
Graphic symbol

<i>E</i>	<i>Operations</i>
0	No change
1	Count

Operation table

Q_i	C_i	C_{i+1}	D_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

HA truth table



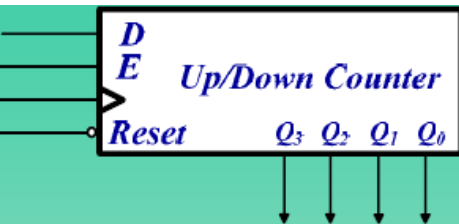
counter schematic

Up-counter or
Down-counter???

$$D_i = Q_i \oplus C_i$$

$$C_{i+1} = Q_i C_i$$

4-bit synchronous binary counter



Graphic symbol

E	D	Operations
0	X	No change
1	0	Count up
1	1	Count down

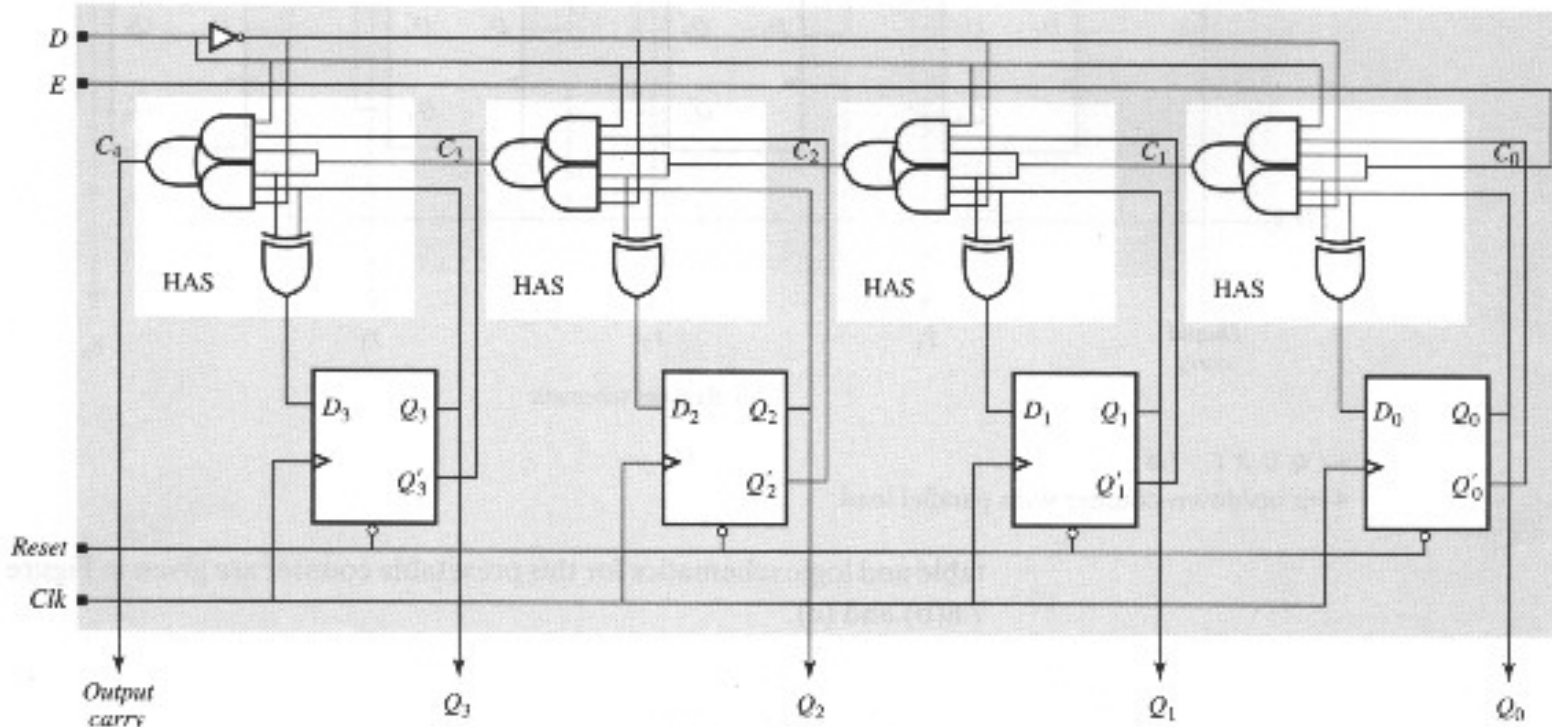
Operation table

E	D	Q_i	C_i	C_{i+1}	D_i
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	1	0	0

HAS truth table

$$D_i = Q_i \oplus C_i$$

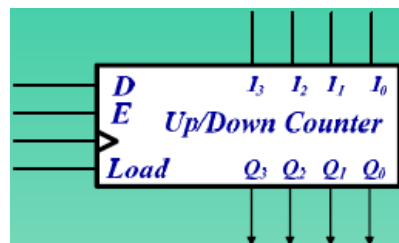
$$C_{i+1} = D'Q_iC_i + DQ_i'C_i$$



Logic schematic

4-bit synchronous binary counter with parallel load

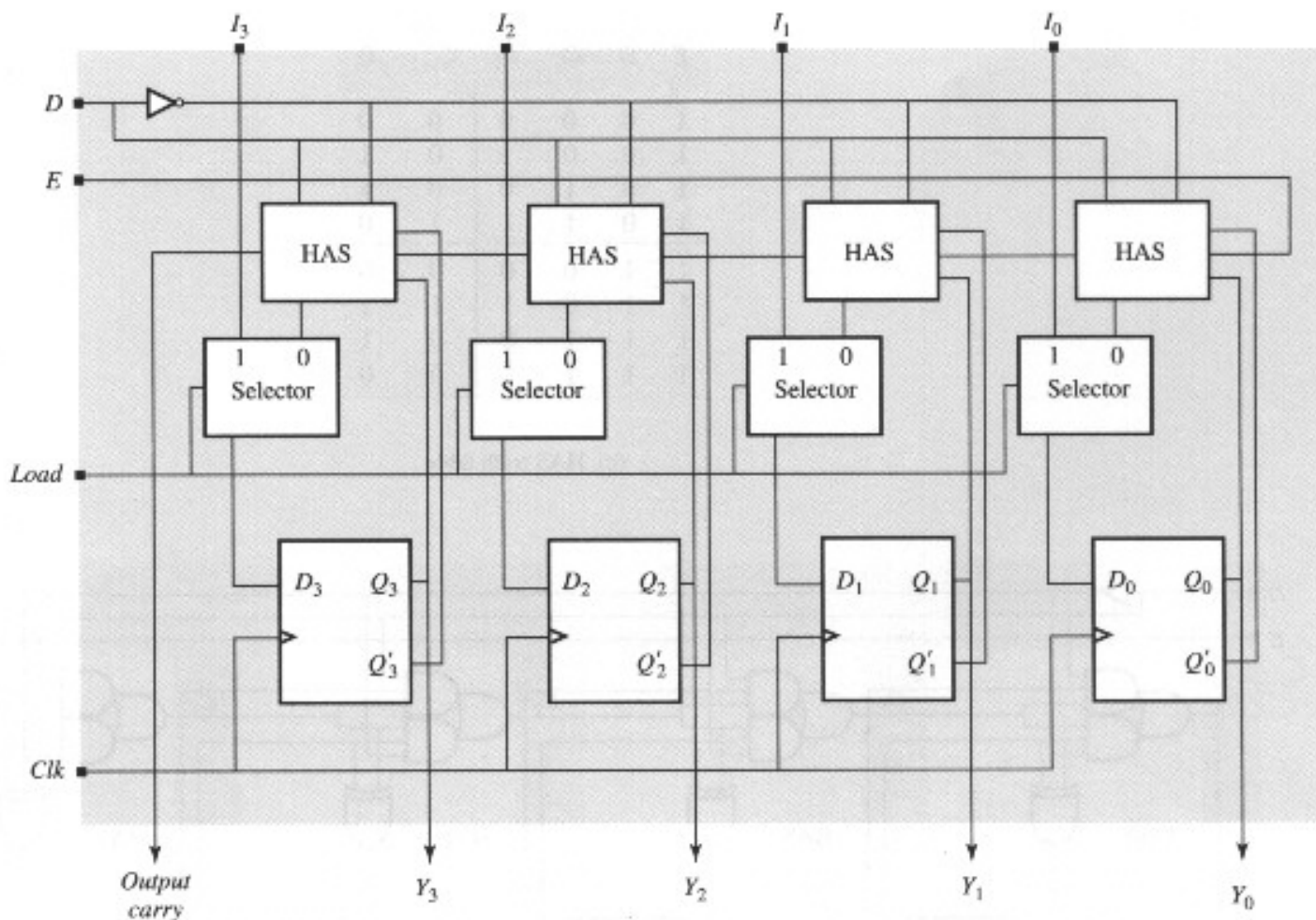
➤ This counter is sometimes called presettable counter



Graphic symbol

Load	E	D	Operations
0	0	X	No change
0	1	0	Count up
0	1	1	Count down
1	X	X	Load the input

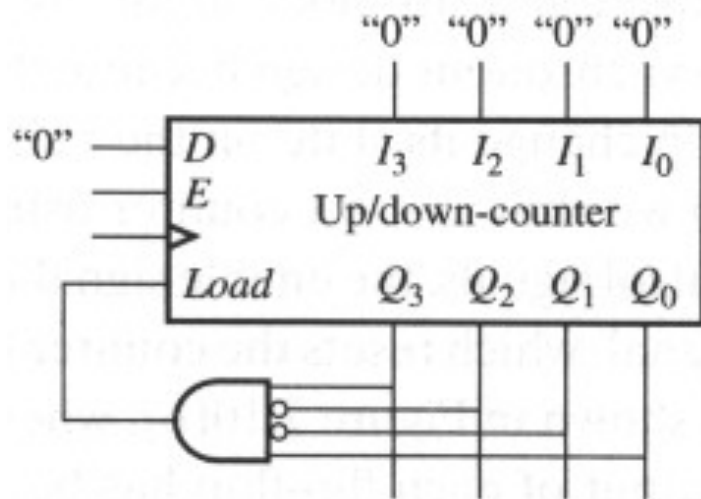
Operation table



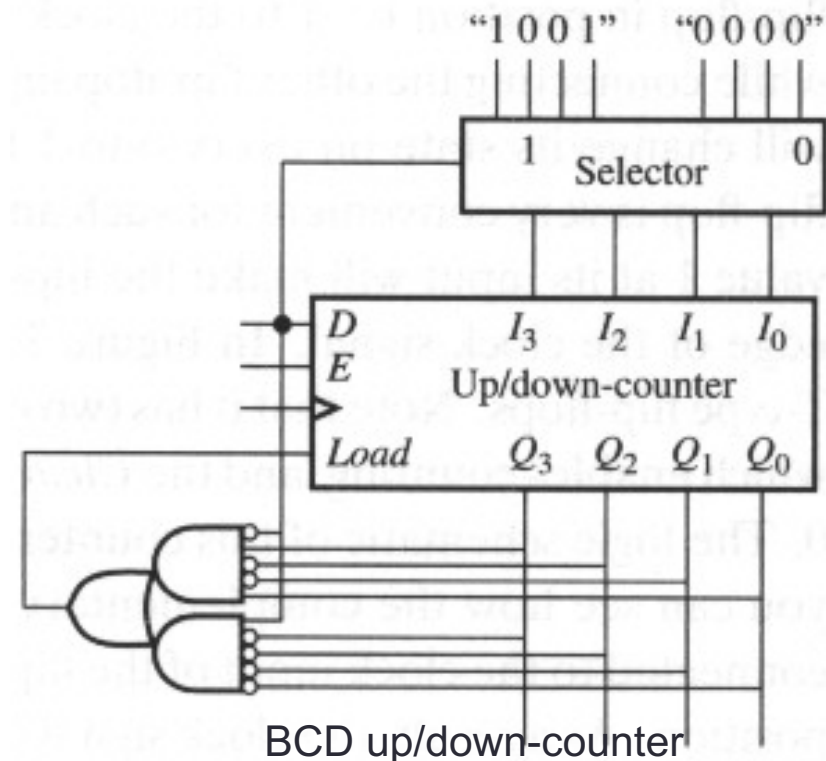
Logic schematic

BCD counters

- Up sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0,
- Down sequence: 0, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 9,



BCD up-counter



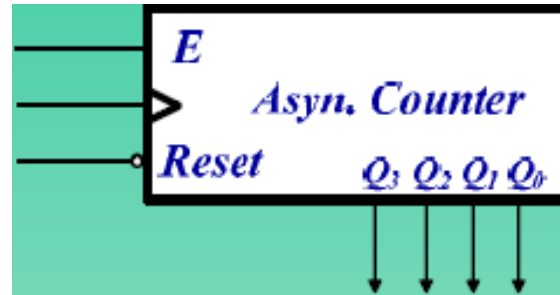
BCD up/down-counter

- Up counter loads 0 when counter content is 9 (1001)
- Up/down counter loads 0 when counter content is 9 (1001) and direction bit D=0
- Up/down counter loads 9 when counter content is 0 (0000) and direction bit D=1

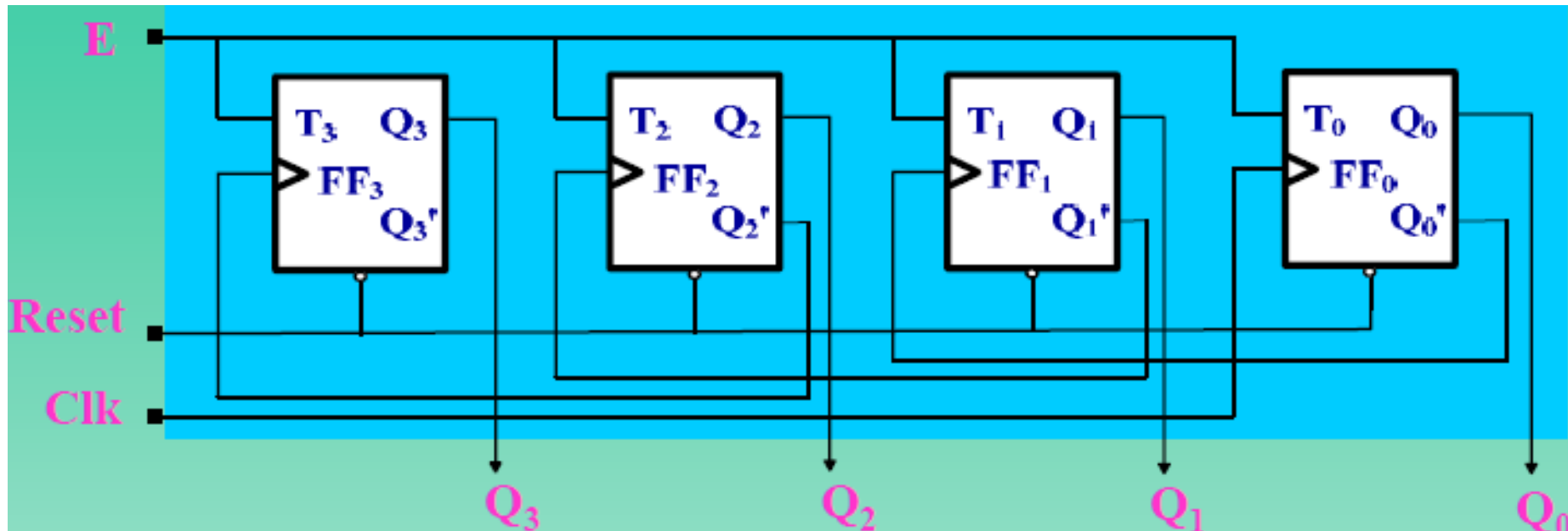
Synchronous vs. Asynchronous counters

- Each FF in **synchronous counters** changes its output at the **same time**.
- FFs' output in **asynchronous counters** change values at **different times**.
- Advantage of asynchronous counters is **simple** and **low cost** (less gates).
- Weakness of asynchronous counters is **longer delays** in comparison with synchronous counters.

4-bit asynchronous up counter

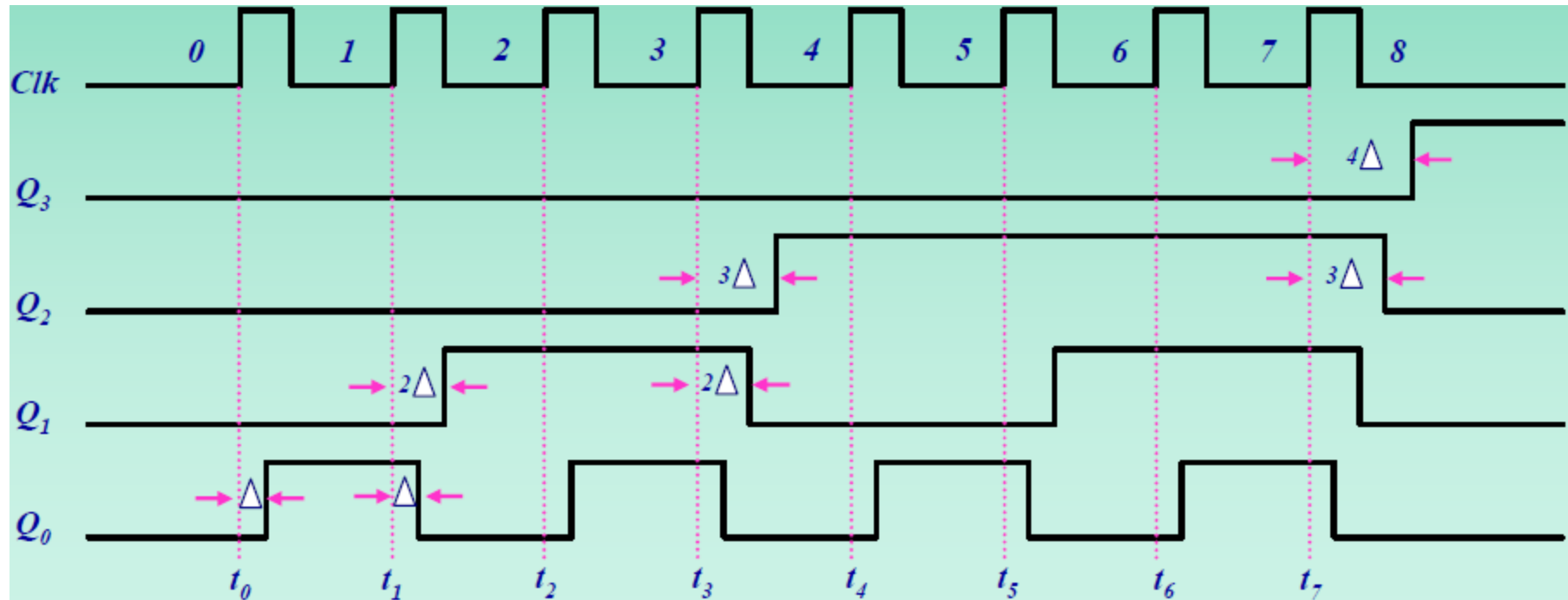


Graphic symbol



Logic schematic

4-bit asynchronous up counter

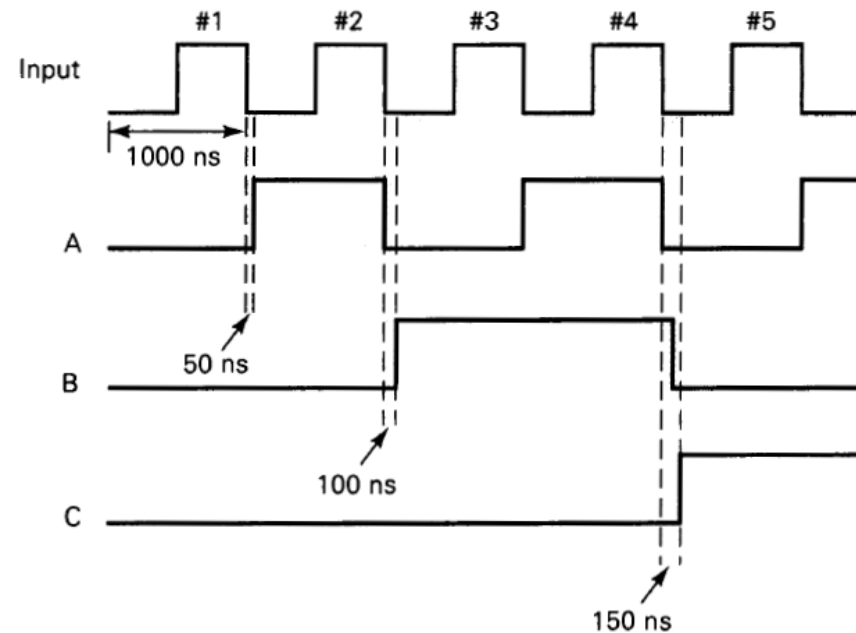


Timing diagram

Propagation Delay in Ripple Counters

- Ripple counters are simple—requiring the fewest components to produce a given operation
 - Cumulative propagation delay can cause problems at high frequencies
- The counter will perform accurately when the period of clock cycle must be longer than the total propagation delay of it.
 - For proper operation: $T_{\text{clock}} \geq N \times t_{\text{pd}}$
 - T_{clock} : the period of clock cycle
 - N : the number of FFs
 - T_{pd} : the propagation delay of a FF (Δ)
 - Maximum frequency: $F_{\text{max}} = 1 / (N \times t_{\text{pd}})$

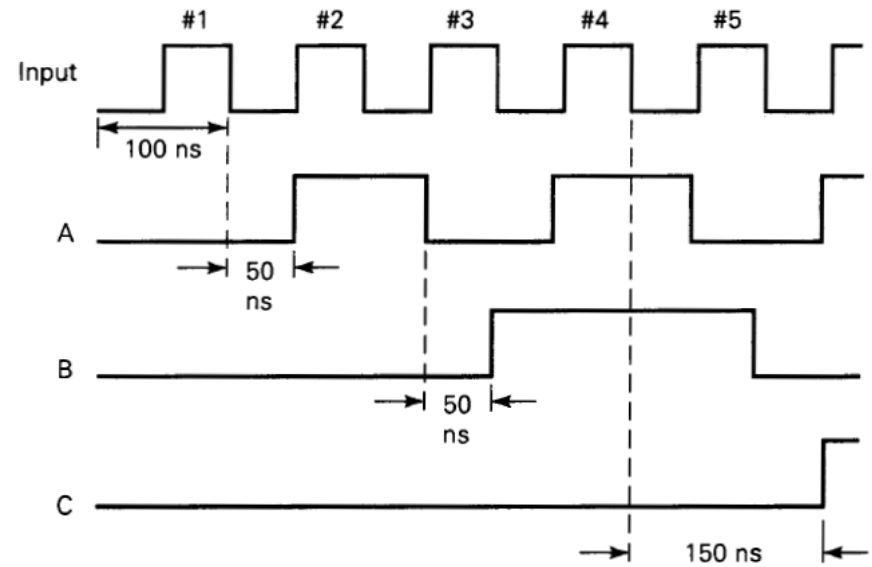
Propagation Delay in Ripple Counters



- $T = 1000 \text{ ns}$
- $t_{pd} = 50 \text{ ns}$

→ $T \geq 3 \times t_{pd}$

Counter operates properly



- $T = 100 \text{ ns}$
- $t_{pd} = 50 \text{ ns}$

→ $T < 3 \times t_{pd}$

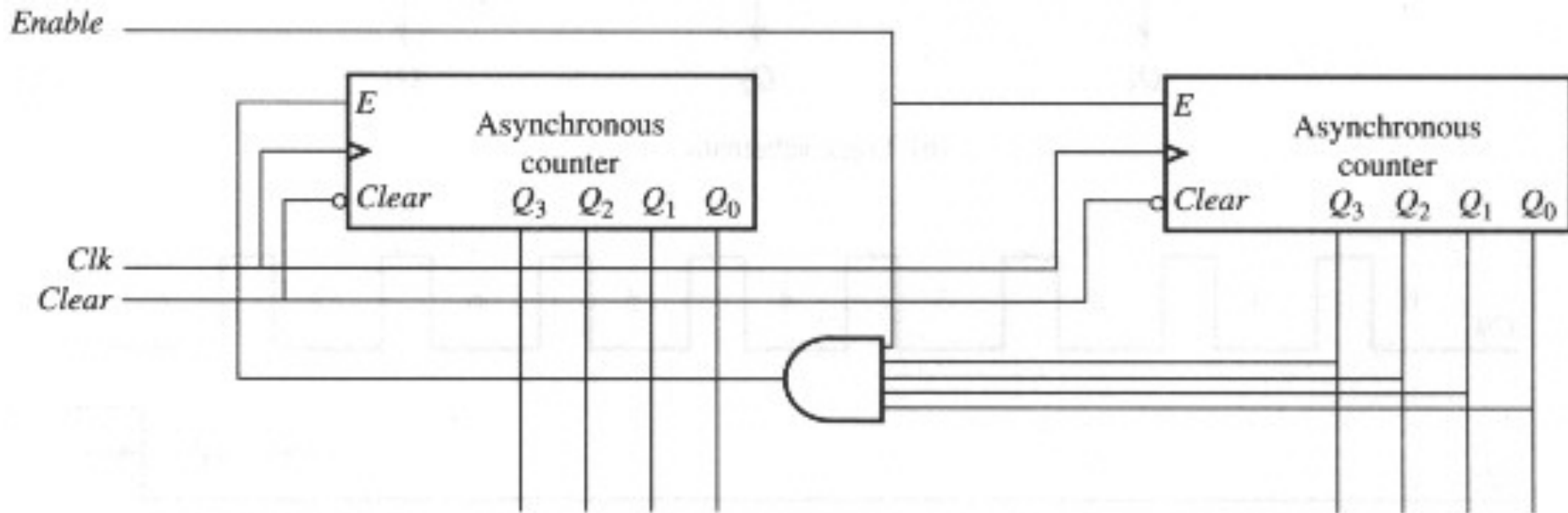
Counter operates wrongly

The CBA = 100
does not occur

8-bit mixed-mode up counter (cont.)

➤ **Mixed-mode counter consists of:**

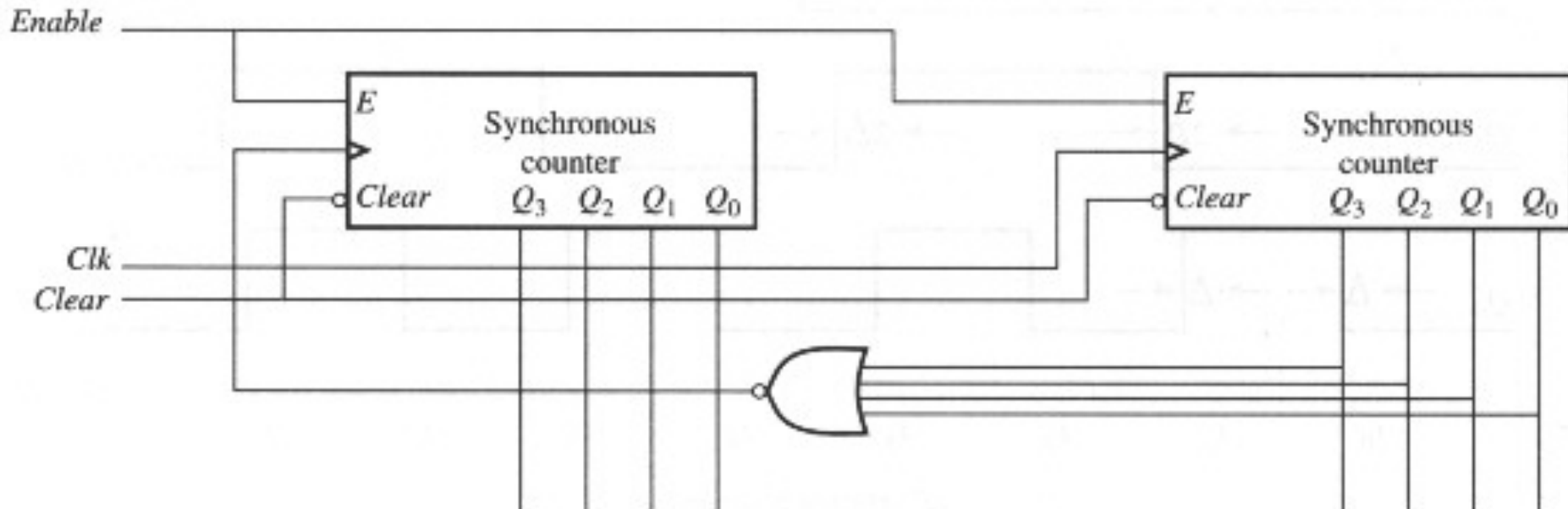
- **synchronous counters with asynchronous slides.**



Synchronous counter with 4-bit asynchronous slices

8-bit mixed-mode up counter

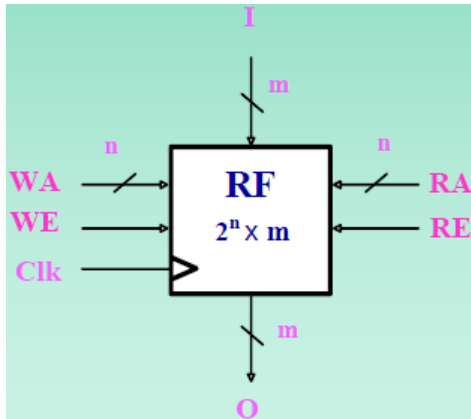
- **Mixed-mode counter consists of:**
 - asynchronous counters with synchronous slices.



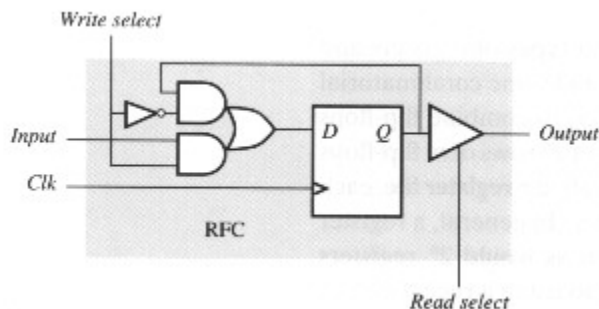
Asynchronous counter with 4-bit synchronous slices

Register-file

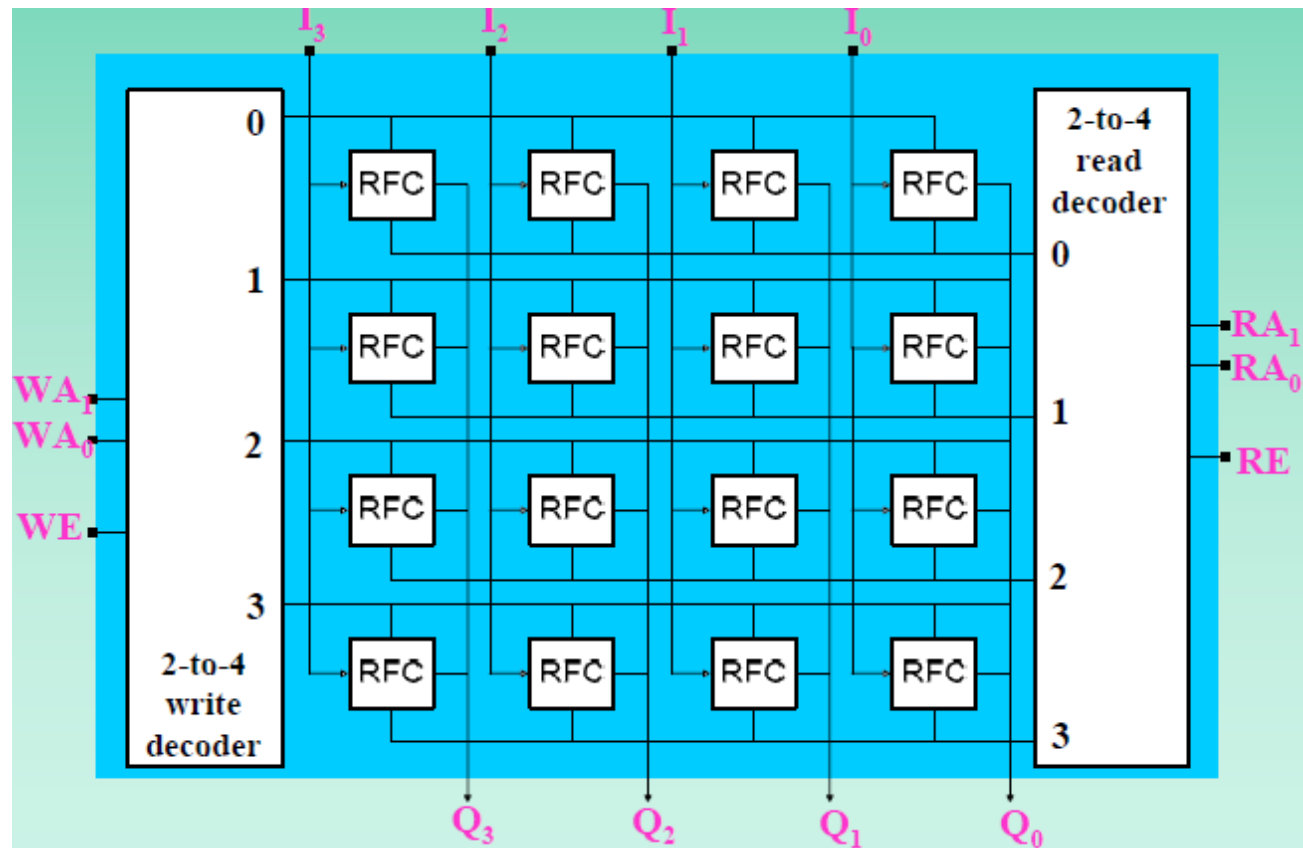
➤ Register-file is used as fast temporary storage



Graphic symbol



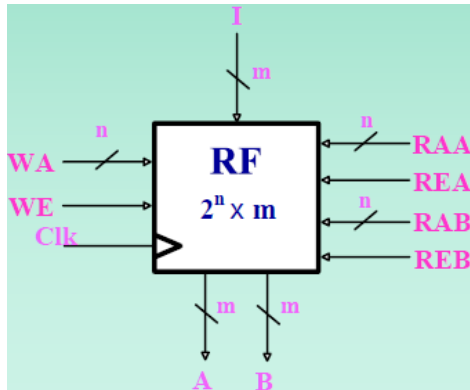
Register-file cell



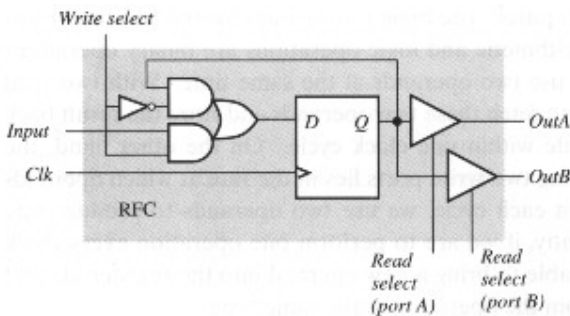
Logic schematic

Register-file with 1 write port and 2 read ports

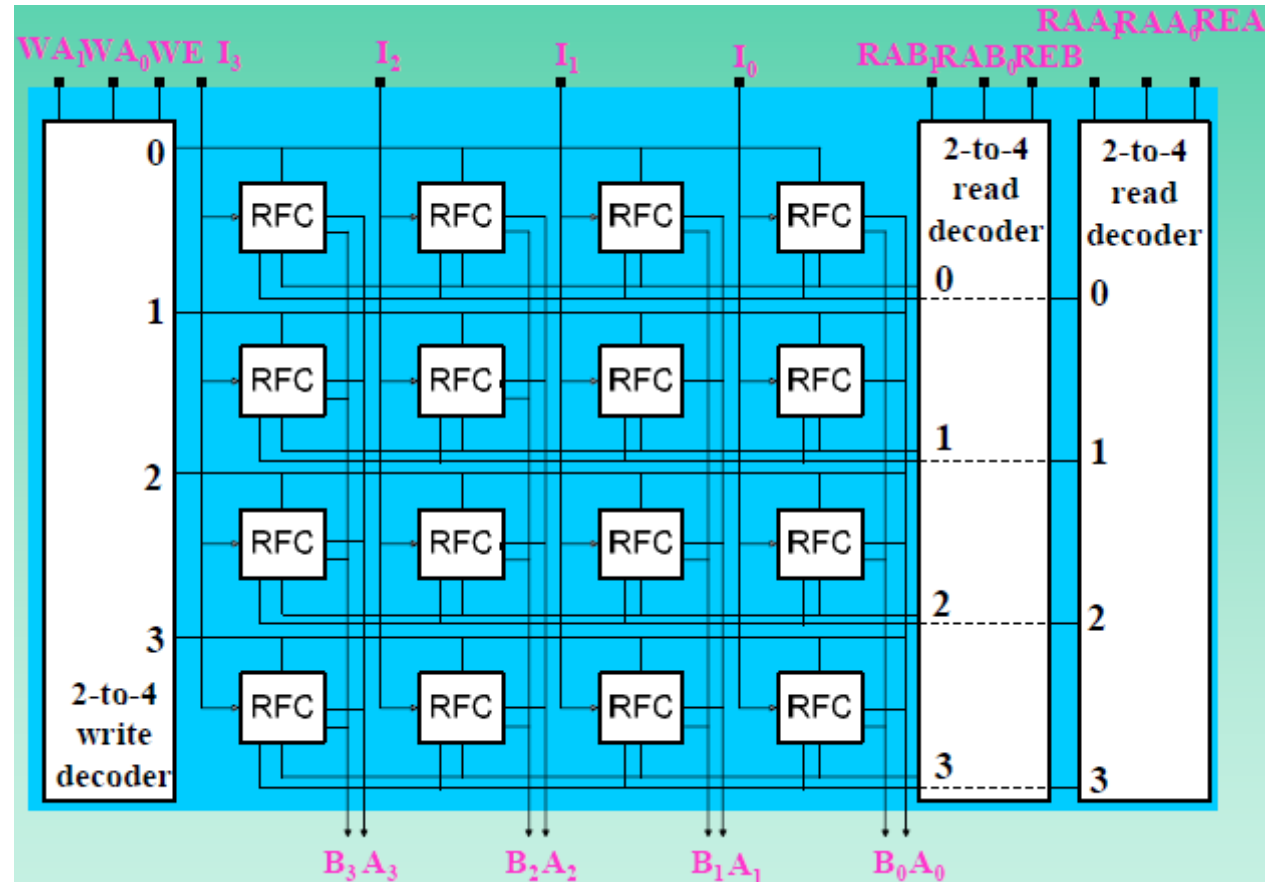
- This register-file is used for reading two operands and writing one result in each clock cycle.



Graphic symbol



Register-file cell



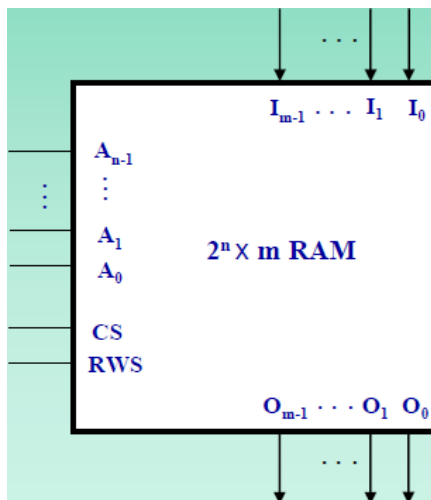
Logic schematic

Random Access Memory (RAM)

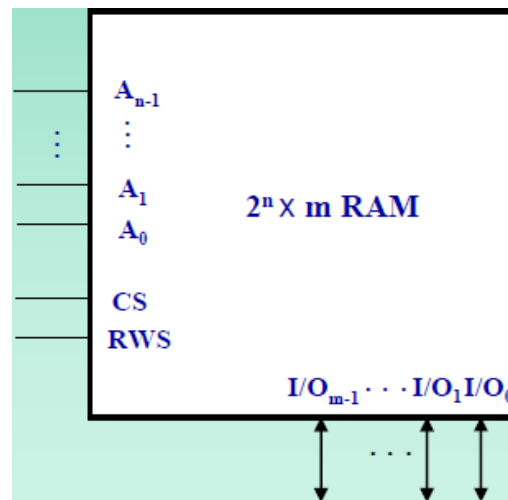
MEMORY ADDRESS		MEMORY CONTENT
Binary	Decimal	
0...000	0	011...0100
0...001	1	011...0100
0...010	2	101...1100
0...011	3	101...0001
0...100	4	011...0101
0...101	5	010...0101
0...110	6	110...0011
0...111	7	101...0001
⋮	⋮	⋮
1...110	2^n-2	000...0010
1...111	2^n-1	111...0110

$\xleftrightarrow{\text{m bits}}$

Memory address
and content



Graphic symbol



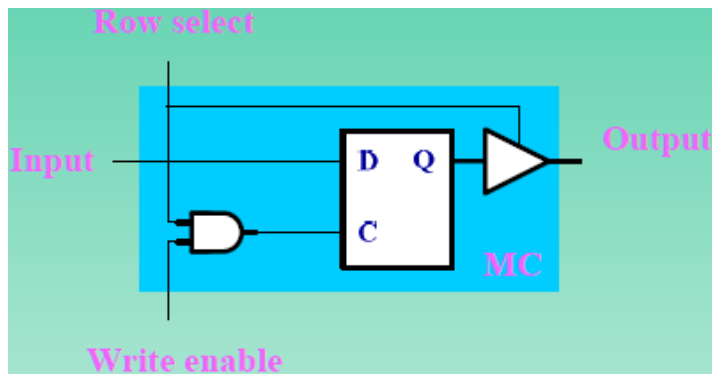
Graphic symbol

CS: Chip Select

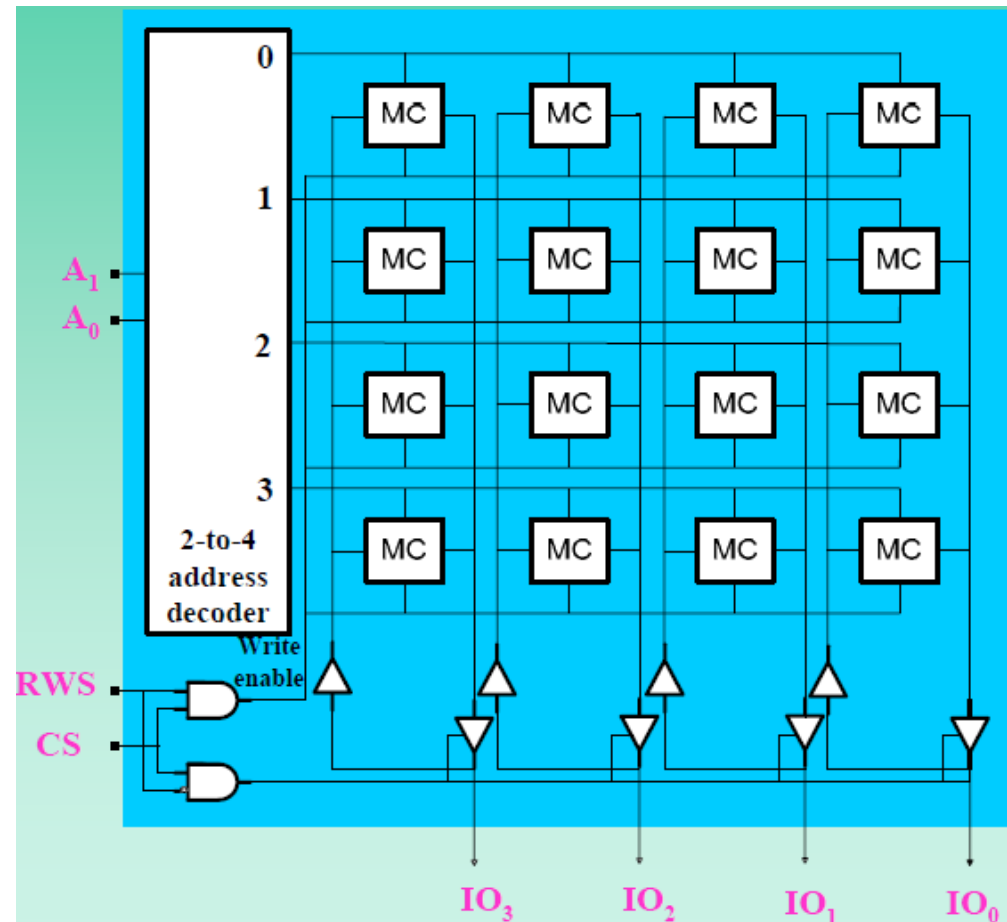
RWS: Read/Write Select
0: Read, 1: Write

RAM organization

- Ram memory cells can be static or dynamic.
- Static RAM do not lose data with time.
- Dynamic RAM must be refreshed.

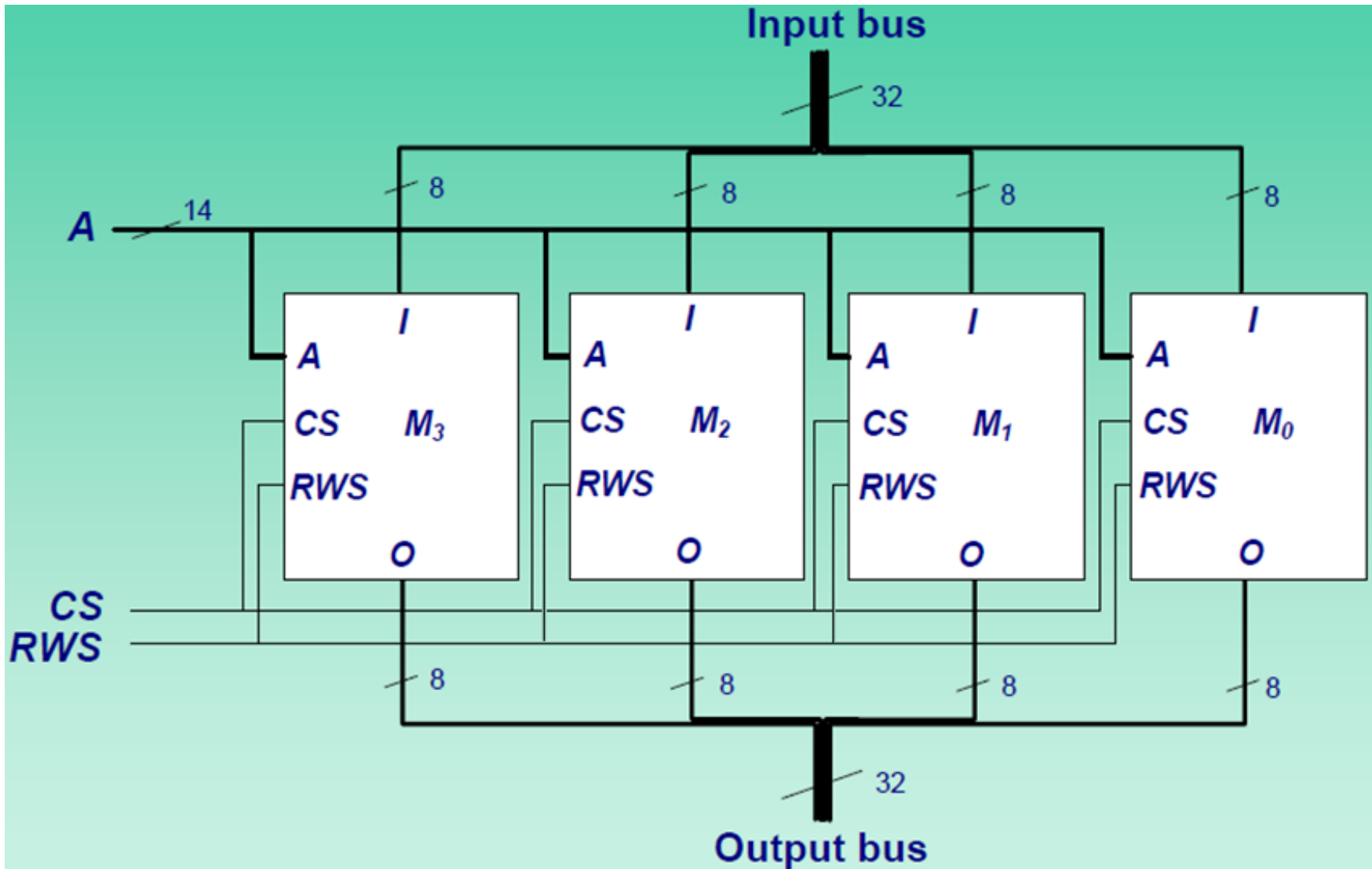


Memory cell

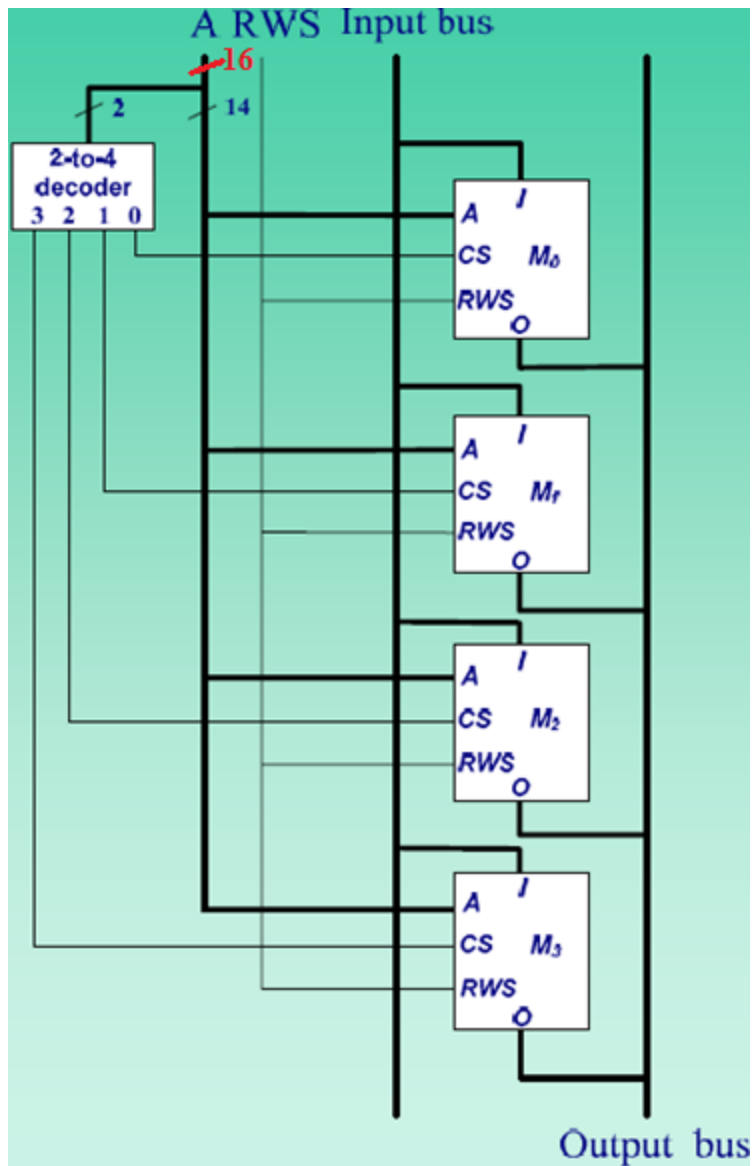


Memory schematic

16K x 32 RAM design with 16K x 8 RAMs (to obtain wider bit widths)



64K x 8 RAM design with 16K x 8 RAMs (to obtain a larger memory)



Addresses

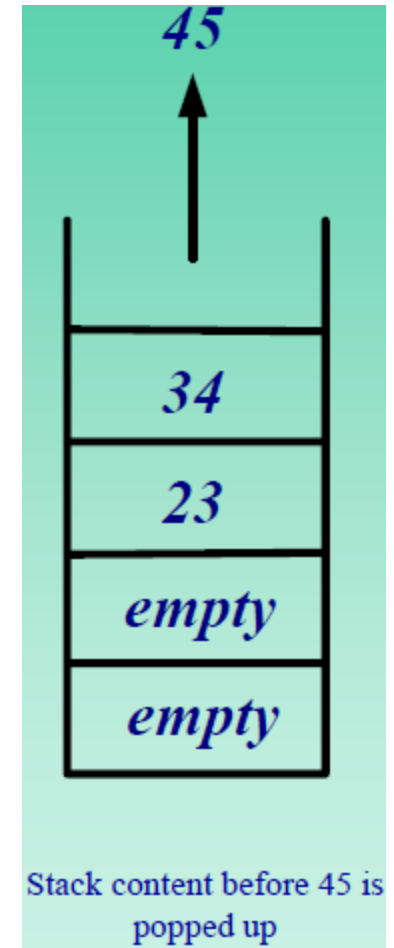
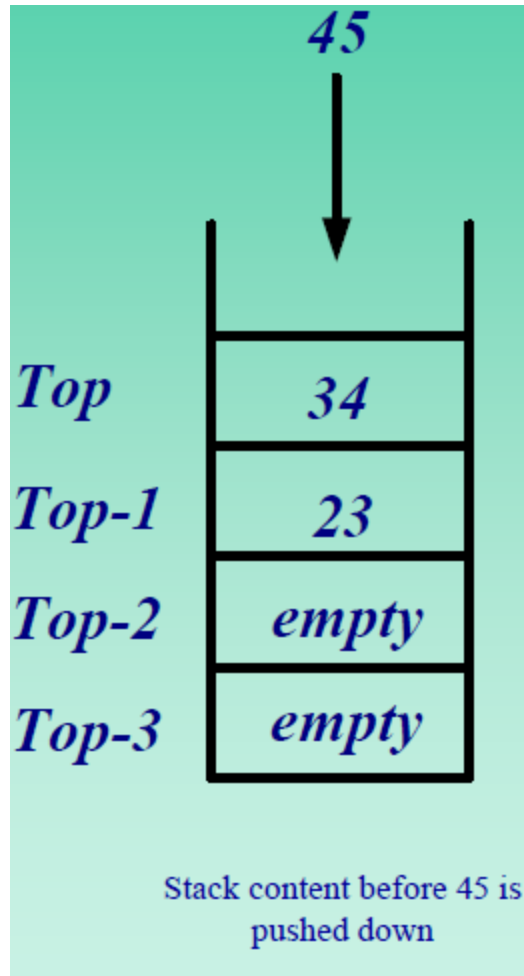
0
 :
 :
 :
 $2^{14}-1$

 2^{14}
 :
 :
 :
 $2^{15}-1$

 2^{15}
 :
 :
 :
 $2^{15}+2^{14}-1$

 $2^{15}+2^{14}$
 :
 :
 :
 $2^{16}-1$

Push-down stack principle



4-Word push-down stack

Push/Pop	Enable	Operations
X	0	No change
0	1	Push
1	1	Pop

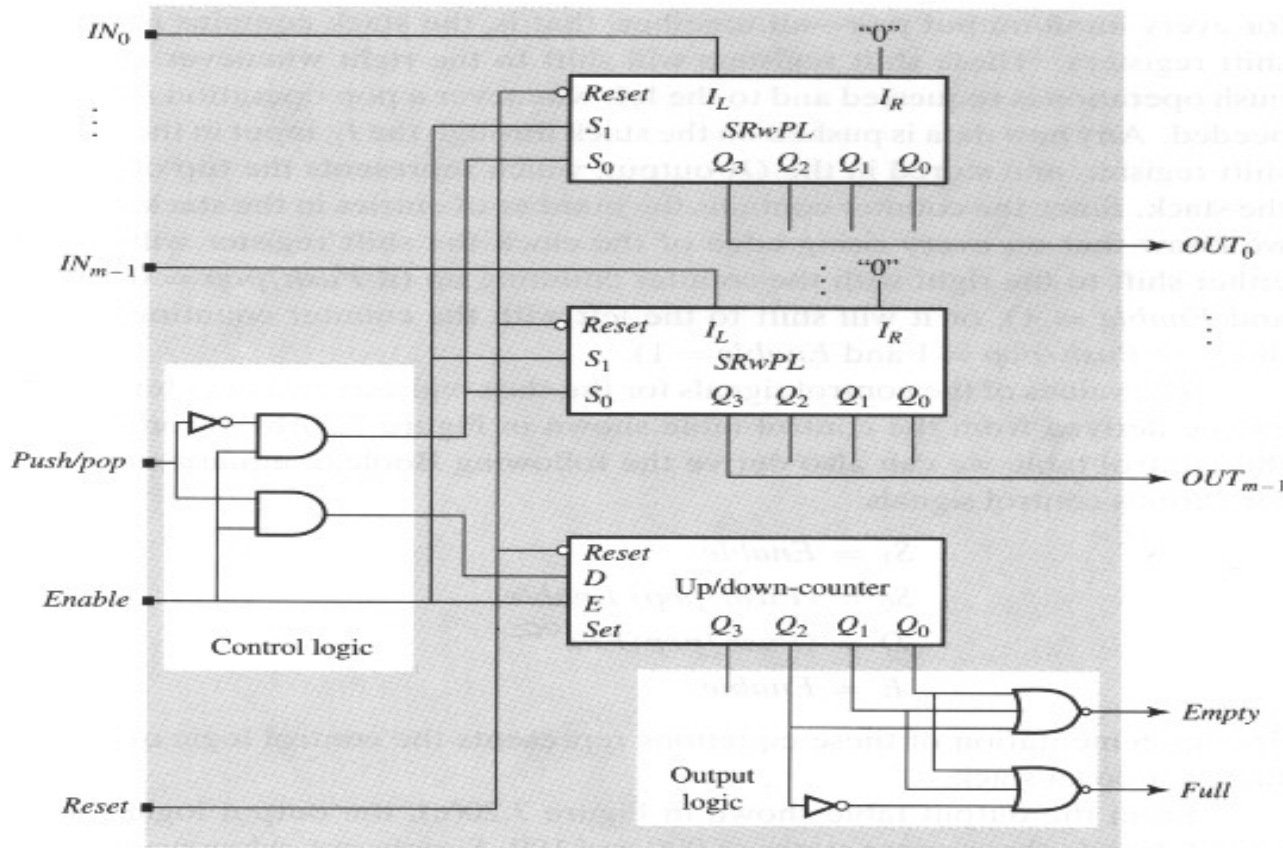
Operation table

		Shift register controls		Counter controls	
Push/Pop Enable		S_1	S_0	D	E
X	0	0	0	X	0
0	1	1	1	0	1
1	1	1	0	1	1

Control table

Counter outputs			Empty	Full
Q_2	Q_1	Q_0		
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	1

Output table

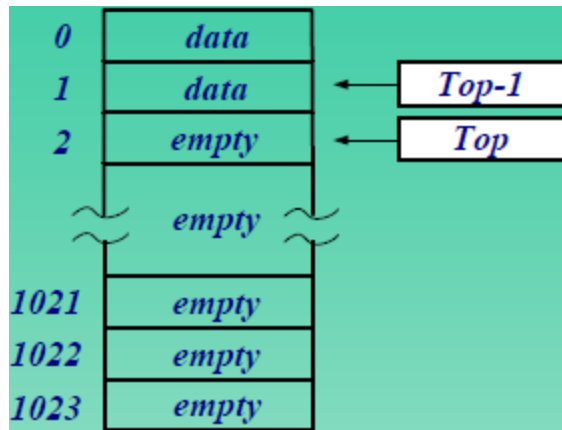


Stack schematic

**Push=Shift right,
count up**

**Pop=Shift left,
count down**

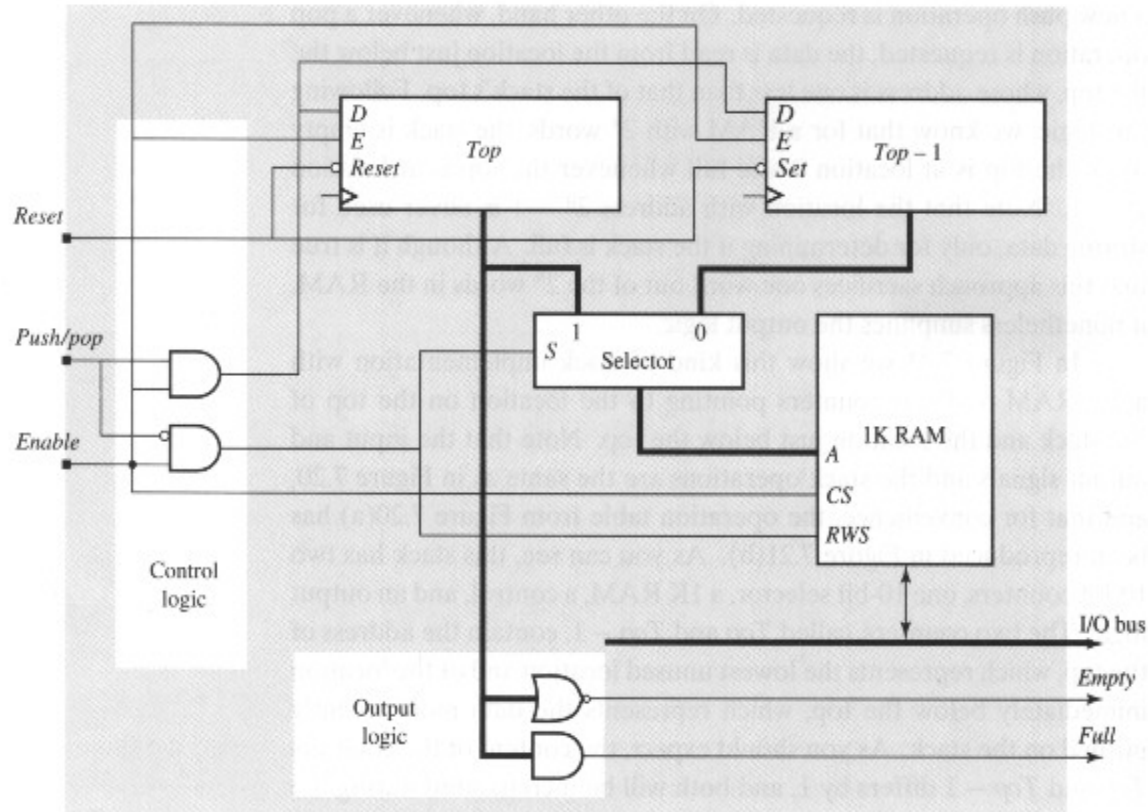
Push-down stack implemented with a 1K RAM



Symbolic design

Push/Pop	Enable	Operations
X	0	No change
0	1	Push
1	1	Pop

Operation table



Push: Data $\rightarrow$ RAM (*TOP*); Increment Top, Top 1
Pop: RAM (*Top-1*) $\rightarrow$ Data; Decrement Top, Top-1
Stack is full when Top=1023
Stack is empty when Top=0
 Location with address 1023 is never loaded

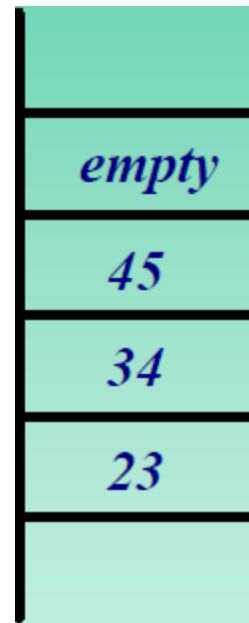
		Selector control	Memory controls		Counter controls	
Push/Pop	Enable	S	CS	RWS	D	E
X	0	X	0	0	X	0
0	1	1	1	1	0	1
1	1	0	1	0	1	1

Control table

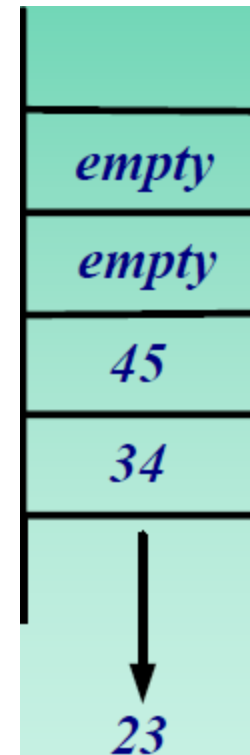
FIFO queue principles



Queue content
before 45 is
stored



Queue content
after 45 is stored



Queue content
after 23 is read

4-word FIFO queue

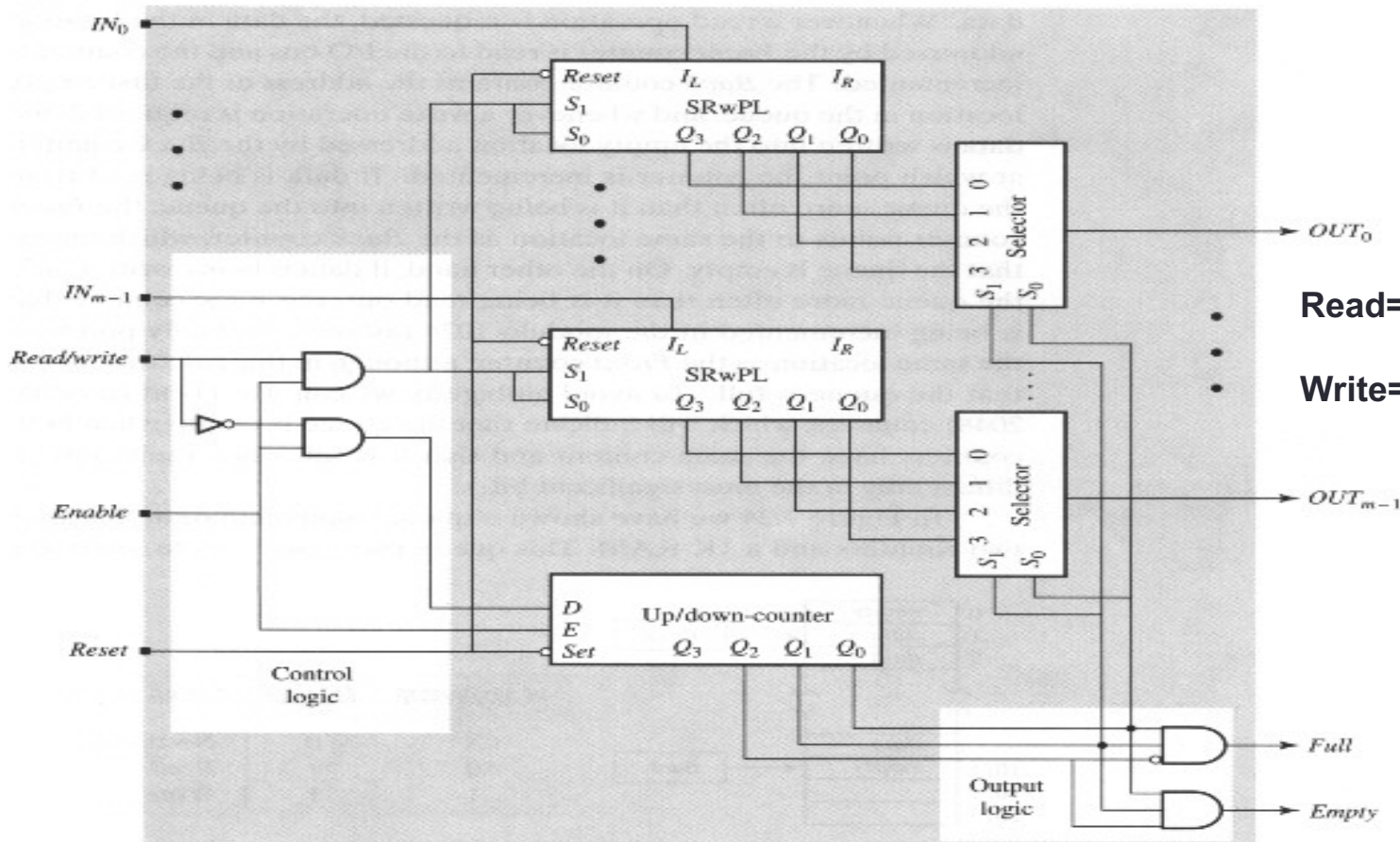
READ/WRITE	ENABLE	OPERATIONS
X	0	No change
0	1	Read
1	1	Write

Operation table

READ/WRITE	ENABLE	S_1	S_0	D	E
X	0	0	0	X	0
0	1	0	0	1	1
1	1	1	1	0	1

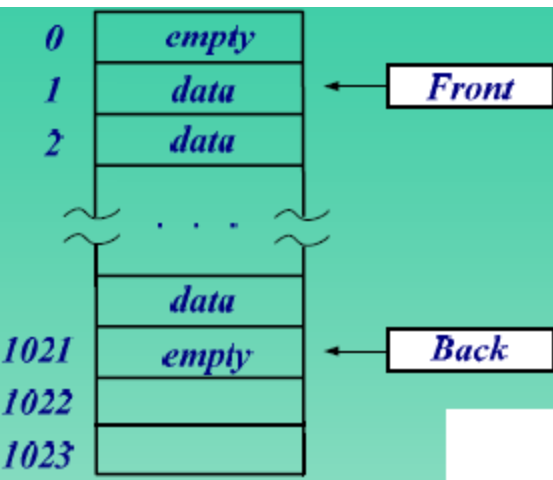
Shift right

Control table



Queue schematic

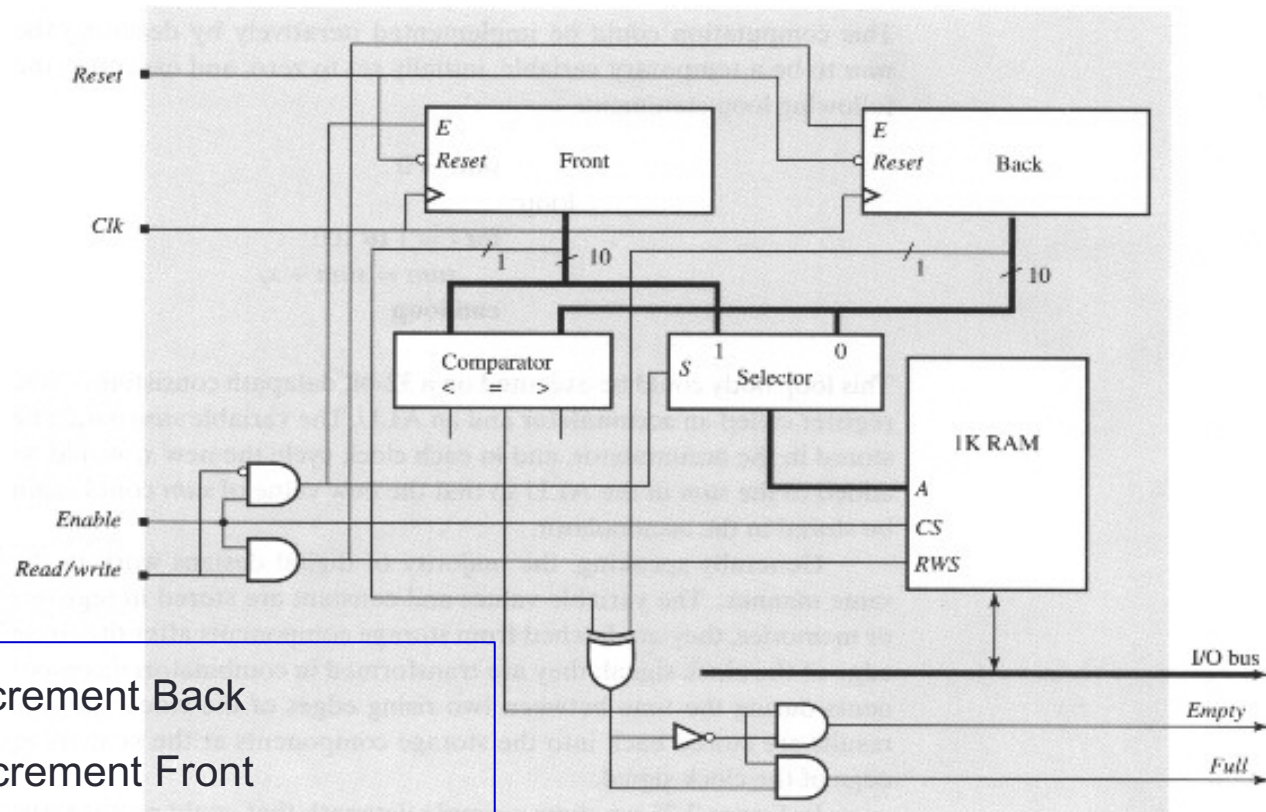
FIFO queue implemented with a 1K RAM



Symbolic design

Read/Write Enable		S	CS	RWS	E (Front)	E (Back)
X	0	X	0	X	0	0
0	1	1	1	0	1	0
1	1	0	1	1	0	1

Control table



Schematic

Read/Write	Enable	Operations
X	0	No change
0	1	Read
1	1	Write

Operation table

Write: Data → RAM (Back); Increment Back

Read: RAM (Front) → Data; Increment Front

Queue is full/empty when Front=Back